Experiment 12: Study and Analyse Container Orchestration using Kubernetes

---

Objective

To understand Kubernetes concepts and perform deployment, scaling, and self-healing of applications using `kubectl`.

Theory

What is Kubernetes?

Kubernetes is an open-source container orchestration platform used to automate deployment, scaling, and management of containerized applications.

Why Kubernetes?

- Industry standard for orchestration
- Supports automatic scaling
- Provides self-healing
- Works across cloud platforms

Core Concepts

Docker Concept	Kubernetes Equivalent	Description
Container	Pod	Smallest unit in Kubernetes
Service	Service	Exposes application
Compose	Deployment	Manages pods
Scaling	ReplicaSet	Maintains desired pod count

Tools Used

- kubectl
- Kubernetes cluster (k3d / Minikube)
- macOS Terminal

Implementation Steps

Step 1: Verify Cluster

```
kubectl get nodes
```

```
Last login: Wed Apr 29 21:10:24 on ttys000
pratiksmac@MacBook-3 ~ % kubectl version --client
Client Version: v1.35.3
Kustomize Version: v5.7.1
pratiksmac@MacBook-3 ~ % minikube version
minikube version: v1.38.1
commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0
pratiksmac@MacBook-3 ~ %
```

```
pratiksmac@MacBook-3 ~ % minikube start --driver=docker --kubernetes-version=v1.30.0
😄  minikube v1.38.1 on Darwin 26.4.1 (arm64)
✨  Using the docker driver based on user configuration
❗  Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
📌  Using Docker Desktop driver with root privileges
👍  Starting "minikube" primary control-plane node in "minikube" cluster
🚜  Pulling base image v0.0.50 ...
💾  Downloading Kubernetes v1.30.0 preload ...
    > preloaded-images-k8s-v18-v1...:  319.81 MiB / 319.81 MiB  100.00% 2.96 Mi
🔥  Creating docker container (CPUs=2, Memory=3072MB) ...
🐳  Preparing Kubernetes v1.30.0 on Docker 29.2.1 ...
🔗  Configuring bridge CNI (Container Networking Interface) ...
🔎  Verifying Kubernetes components...
    ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟  Enabled addons: default-storageclass, storage-provisioner

❗  /opt/homebrew/bin/kubectl is version 1.35.3, which may have incompatibilities with Kubernetes 1.30.0.
    ▪ Want kubectl v1.30.0? Try 'minikube kubectl -- get pods -A'
🏄  Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
pratiksmac@MacBook-3 ~ %
```

Step 2: Create Deployment File

```
nano wordpress-deployment.yaml
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress
spec:
  replicas: 2
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
```

```
      - name: wordpress
        image: wordpress:latest
        ports:
        - containerPort: 80
```

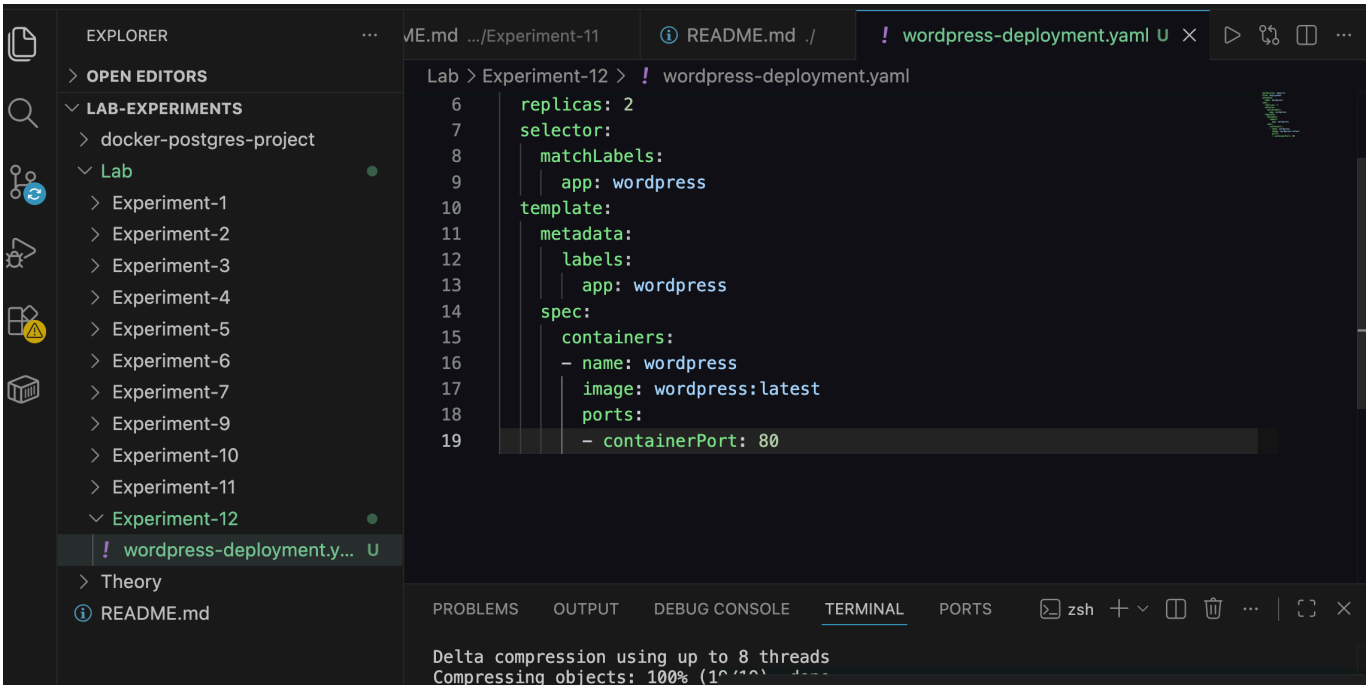
Step 3: Apply Deployment

```
kubectl apply -f wordpress-deployment.yaml
```

```
pratiksmac@MacBook-3 ~ % kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   58s   v1.30.0
pratiksmac@MacBook-3 ~ %
```

Step 4: Verify Pods

```
kubectl get pods
```



Step 5: Create Service File

```
nano wordpress-service.yaml
```

```yaml
apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
spec:
  type: NodePort
  selector:
    app: wordpress
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30007
```

Step 6: Apply Service

```
kubectl apply -f wordpress-service.yaml
```

```
pratiksmac@MacBook-3 Experiment-12 % kubectl apply -f wordpress-deployment.yaml
deployment.apps/wordpress created
pratiksmac@MacBook-3 Experiment-12 % kubectl get pods
NAME                          READY   STATUS              RESTARTS   AGE
wordpress-5bfc95c6dc-cck59    0/1     ContainerCreating   0          18s
wordpress-5bfc95c6dc-r5kp2    0/1     ContainerCreating   0          18s
pratiksmac@MacBook-3 Experiment-12 % kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
wordpress-5bfc95c6dc-cck59    1/1     Running   0          2m3s
wordpress-5bfc95c6dc-r5kp2    1/1     Running   0          2m3s
pratiksmac@MacBook-3 Experiment-12 %
```

Step 7: Verify Service

```
kubectl get svc
```

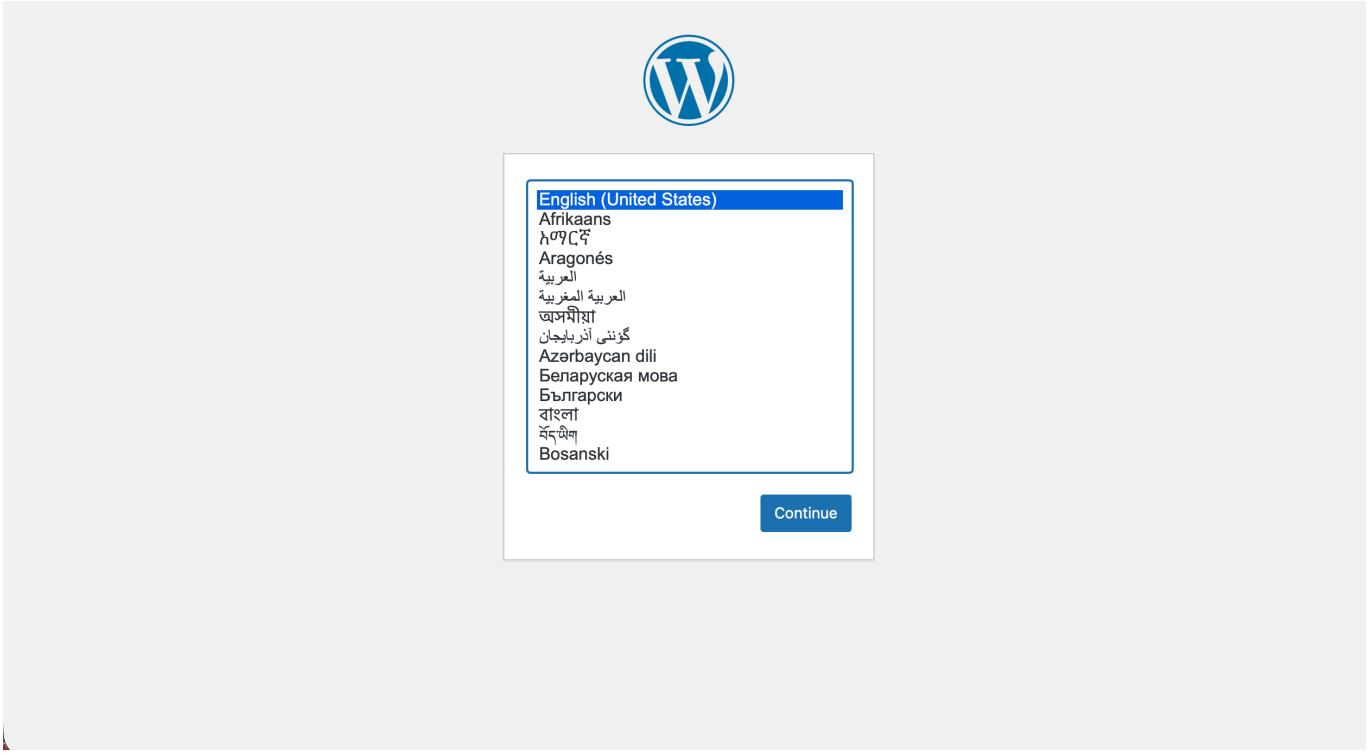
```
pratiksmac@MacBook-3 Experiment-12 % kubectl get svc
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP   7m20s
pratiksmac@MacBook-3 Experiment-12 % kubectl apply -f wordpress-service.yaml
service/wordpress-service created
pratiksmac@MacBook-3 Experiment-12 % kubectl get svc
NAME                 TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
kubernetes           ClusterIP   10.96.0.1      <none>        443/TCP        7m46s
wordpress-service    NodePort    10.108.66.38   <none>        80:30007/TCP   7s
pratiksmac@MacBook-3 Experiment-12 %
```

Step 8: Access Application

```
kubectl port-forward service/wordpress-service 8082:80
```

Open:

```
http://localhost:8082
```



Step 9: Scale Deployment

```
kubectl scale deployment wordpress --replicas=4
```

Step 10: Verify Scaling

```
kubectl get pods
```

```
pratiksmac@MacBook-3 Experiment-12 % kubectl scale deployment wordpress --replicas=4
deployment.apps/wordpress scaled
pratiksmac@MacBook-3 Experiment-12 % kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
wordpress-5bfc95c6dc-cck59    1/1     Running   0          8m53s
wordpress-5bfc95c6dc-r5kp2    1/1     Running   0          8m53s
wordpress-5bfc95c6dc-vshwk    1/1     Running   0          2m10s
wordpress-5bfc95c6dc-wqnnr    1/1     Running   0          2m10s
pratiksmac@MacBook-3 Experiment-12 %
```

Step 11: Test Self-Healing

```
kubectl get pods
kubectl delete pod <pod-name>
kubectl get pods
```

```
pratiksmac@MacBook-3 Experiment-12 % kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
wordpress-5bfc95c6dc-cck59    1/1     Running   0          10m
wordpress-5bfc95c6dc-r5kp2    1/1     Running   0          10m
wordpress-5bfc95c6dc-vshwk    1/1     Running   0          3m41s
wordpress-5bfc95c6dc-wqnnr    1/1     Running   0          3m41s
pratiksmac@MacBook-3 Experiment-12 % kubectl delete pod wordpress-5bfc95c6dc-cck59
pod "wordpress-5bfc95c6dc-cck59" deleted from default namespace
pratiksmac@MacBook-3 Experiment-12 % kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
wordpress-5bfc95c6dc-kgnnk    1/1     Running   0          11s
wordpress-5bfc95c6dc-r5kp2    1/1     Running   0          11m
wordpress-5bfc95c6dc-vshwk    1/1     Running   0          4m38s
wordpress-5bfc95c6dc-wqnnr    1/1     Running   0          4m38s
pratiksmac@MacBook-3 Experiment-12 %
```

Step 12: Cleanup

```
kubectl delete -f wordpress-deployment.yaml
kubectl delete -f wordpress-service.yaml
```

Result

- Deployment created successfully
- Service exposed the application
- Scaling increased pods from 2 to 4
- Kubernetes automatically recreated deleted pods

Conclusion

Kubernetes simplifies container orchestration by providing automated deployment, scaling, and self-healing capabilities, making it a powerful tool for modern applications.

Key Takeaways

- Deployment manages pods
- Service exposes applications
- Scaling improves performance
- Self-healing ensures reliability

Key Learning

> Kubernetes ensures desired state by automatically managing containers, scaling applications, and recovering from failures.